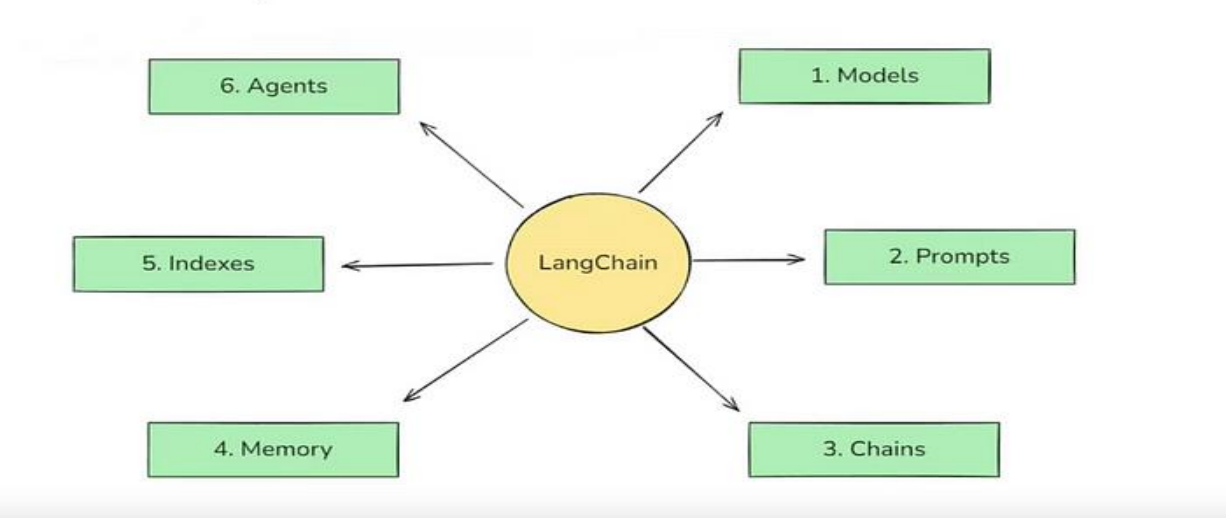
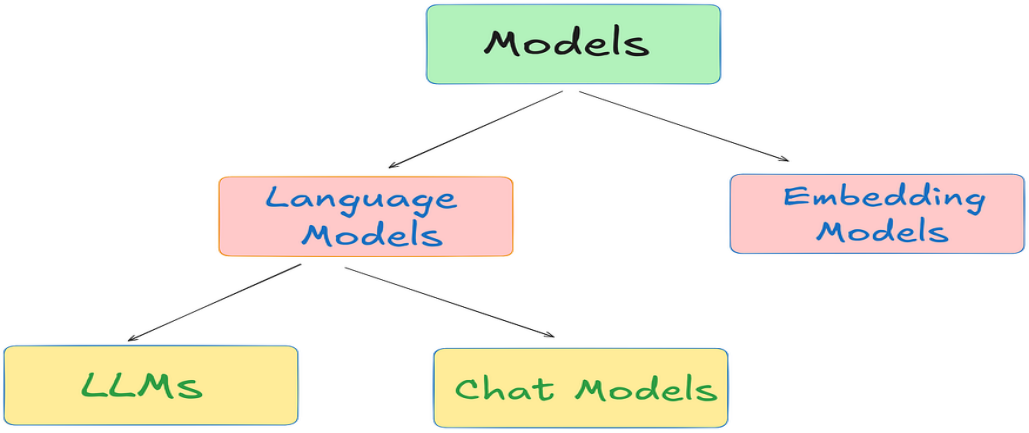
“LangChain Notes (2)”

In the previous blog, we explored what LangChain is, why it’s useful, and gave an overview of its six core components. Now in this Part-2 blog, we’ll dive deeper into one of the most important parts of LangChain — the Model component. This is where the actual communication with language models happens. We’ll break down how it works, the different types of models you can use, and how to customize them to suit your application needs.



The Model component in LangChain consists of two main types: Language Models and Embedding Models. Language Models are used to generate text, while Embedding Models convert text into vector representations for tasks like semantic search. Within Language Models, there are two subtypes: LLMs (Large Language Models) and Chat Models. The key difference is that Chat Models are fine-tuned on human conversations, making them better at interactive communication. As a result, Chat Models are now more commonly used than traditional LLMs.



In this blog series, we'll explore each type of model in detail, one by one.
Difference between LLMs and Chat Models :

Feature	LLMs (Base Models)	Chat Models (Instruction-Tuned)
Purpose	Free-form text generation	Optimized for multi-turn conversations
Training Data	General text corpora (books, articles)	Fine-tuned on chat datasets (dialogues, user-assistant conversations)
Memory & Context	No built-in memory	Supports structured conversation history
Role Awareness	No understanding of "user" and "assistant" roles	Understands "system", "user", and "assistant" roles
Example Models	GPT-3, Llama-2-7B, Mistral-7B, OPT-1.3B	GPT-4, GPT-3.5-turbo, Llama-2-Chat, Mistral-Instruct, Claude
Use Cases	Text generation, summarization, translation, creative writing, code generation	Conversational AI, chatbots, virtual assistants, customer support, AI tutors

Since we've started discussing LangChain models, let's begin by setting up our environment. First, choose a location on your system where you'd like to run the experiment. Create a folder there, set up a virtual environment inside it, and then install the following dependencies:

```
# LangChain Core
langchain
langchain-core

# OpenAI Integration
langchain-openai
openai

# Anthropic Integration
langchain-anthropic

# Google Gemini (PaLM) Integration
langchain-google-genai
google-generativeai

# Hugging Face Integration
```

```
langchain-huggingface
transformers
huggingface-hub

# Environment Variable Management
python-dotenv

# Machine Learning Utilities
numpy
scikit-learn
```

Now, let's organize our project by creating three separate folders: one for **LLMs**, one for **Chat Models**, and one for **Embedding Models**. In this blog, we'll explore both **paid APIs** (such as OpenAI, Google Gemini, and Anthropic) and **free APIs** (like Hugging Face). To use the paid APIs, you'll need an active subscription. If you don't have one, no worries — you can still follow along using the free APIs.
Next, create a .env file and securely store all your API keys there for easy access and better security.

```
# .env file
OPENAI_API_KEY = "Your Key"
ANTHROPIC_API_KEY = " "
GOOGLE_API_KEY = " "
HUGGINGFACEHUB_ACCESS_TOKEN = " "
```

Let's begin with the LLMs model first.

```
from langchain_openai import OpenAI
from dotenv import load_dotenv
# load_dotenv load your API key from .env file.
load_dotenv()
# creating OpenAI object and passing the desired model which you want to use.
llm = OpenAI(model='gpt-3.5-turbo-instruct')

# invoke function helps to communicate with the model
result = llm.invoke("What is the capital of India")

print(result)
```

Now, let's move on to the same setup process for Chat Models.

```
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
load_dotenv()
model = ChatOpenAI(model='gpt-4')
result = model.invoke("What is the capital of India")
print(result.content)
```

When working with Chat Models, you'll notice that the output isn't just the plain answer. Instead, it includes a lot of additional metadata. For example, along with the actual response content, you'll also see fields like the number of prompt tokens, completion tokens, and other detailed information.

This is one of the key behavioral differences between Chat Models and standard LLMs. Typically, if you're only interested in the final answer, you don't print the entire result object. Instead, you extract just the "content" field from the response. That's where the actual answer resides.

So when you run the code and access only the content field, the model gives you a clean, straightforward answer like:
"The capital of India is New Delhi."

This approach helps you avoid printing unnecessary metadata and focus only on what matters.

One of the parameter that plays a crucial role while working with language models is temperature. Simply put, **temperature** controls the creativity of the model's responses.

Technically, it's a parameter that adjusts the **randomness** of the output. A lower temperature value (e.g., between 0 and 0.3) makes the responses more deterministic and predictable. This is great when you're looking for factual answers, doing math, or writing code.

On the other hand, a higher temperature value (e.g., 1.0 to 1.5) increases randomness, making the responses more diverse, creative, and sometimes even unpredictable—perfect for tasks like storytelling, writing jokes, or brainstorming ideas.

Here's a quick guide for choosing the right temperature:

- 0 to 0.3 → Best for deterministic tasks like coding or math.
- 0.5 to 0.7 → Good for general Q&A or explanations.
- 0.9 to 1.2 → Use when working on creative tasks like writing or storytelling.

- 1.5+ → Ideal for maximum randomness and open-ended brainstorming.

Another important parameter while working with language models is max completion tokens.

This parameter allows you to control the length of the model's response. You can think of tokens roughly as words. So by setting max completion tokens, you're essentially telling the model how long the output should be — for example, 10 words, 50 words, or 100 words.

Why is this useful?

Especially when using paid APIs like OpenAI, Anthropic, or Gemini, you are charged based on the number of tokens used in both the input and the output. If you visit OpenAI's pricing page, you'll notice the cost is mentioned per million tokens. That means the more tokens you generate, the more you pay.

As a developer, you might want to limit the response size to keep costs in check or to make the output concise for a specific use case. For instance, if you only want a short summary or answer, you can set max_completion_tokens = 10, and the model will generate at most 10 tokens in the response.

```
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
load_dotenv()
model = ChatOpenAI(model='gpt-4', temperature=1.5, max_completion_tokens=10)
result = model.invoke("Write a 5 line poem on cricket")
print(result.content)
```

Now, instead of using the OpenAI API key, let's use Anthropic.

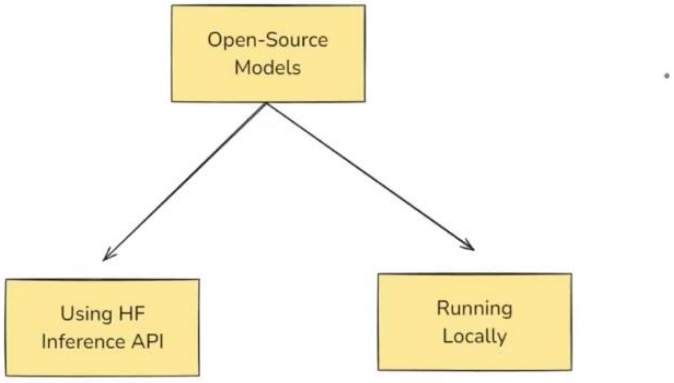
```
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv
load_dotenv()
model = ChatAnthropic(model='claude-3-5-sonnet-20241022')
result = model.invoke('What is the capital of India')
print(result.content)
```

and if you use Google Gemini, then...

```
from langchain_google_genai import ChatGoogleGenerativeAI
from dotenv import load_dotenv
load_dotenv()
```

```
model = ChatGoogleGenerativeAI(model='gemini-1.5-pro')
result = model.invoke('What is the capital of India')
print(result.content)
```

Open Source Models :



If you're looking for a free version, you'll need to create an API key from Hugging Face and then use the following code:

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv
load_dotenv()
llm = HuggingFaceEndpoint(
   repo_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
   task="text-generation"
)

model = ChatHuggingFace(llm=llm)

result = model.invoke("What is the capital of India")

print(result.content)
```

The above code is used when the model is not on your local system. Now, let's look at the code for loading the model on your own machine:

```
from langchain_huggingface import ChatHuggingFace, HuggingFacePipeline
llm = HuggingFacePipeline.from_model_id(
   model_id='TinyLlama/TinyLlama-1.1B-Chat-v1.0',
   task='text-generation',
   pipeline_kwargs=dict(
      temperature=0.5,
```

```
        max_new_tokens=100
    )
)
model = ChatHuggingFace(llm=llm)
result = model.invoke("What is the capital of India")
print(result.content)
```

The above code first downloads the model to your local system and then performs inference for your query. Since CPUs are not optimized for LLM inference, I chose the Tiny LLaMA model for better performance.

Now, it's time to start experimenting with embedding models.

Embedding Models :

```
from langchain_openai import OpenAIEmbeddings
from dotenv import load_dotenv
load_dotenv()
embedding = OpenAIEmbeddings(model='text-embedding-3-large', dimensions=32)
result = embedding.embed_query("Delhi is the capital of India")

print(str(result))
```

The above code is for creating embeddings of a single sentence. Now, suppose we have a paragraph or document:

```
from langchain_openai import OpenAIEmbeddings
from dotenv import load_dotenv
load_dotenv()
embedding = OpenAIEmbeddings(model='text-embedding-3-large', dimensions=32)
documents = [
    "Delhi is the capital of India",
    "Kolkata is the capital of West Bengal",
    "Paris is the capital of France"
]
result = embedding.embed_documents(documents)
print(str(result))
```

Opensource Embedding models:

```
from langchain_huggingface import HuggingFaceEmbeddings
embedding = HuggingFaceEmbeddings(model_name='sentence-transformers/all-
```

```
MiniLM-L6-v2')
documents = [
   "Delhi is the capital of India",
   "Kolkata is the capital of West Bengal",
   "Paris is the capital of France"
]
vector = embedding.embed_documents(documents)
print(str(vector))
```

Summary :

In this blog, we dive into LangChain's Model component, exploring its two main types: Language Models (LLMs) and Embedding Models. LLMs generate text, while Embedding Models convert text into vector representations for tasks like semantic search. We compare traditional LLMs and Chat Models, highlighting their differences in interactivity and use cases. The blog also walks through the setup process for integrating various APIs (OpenAI, Anthropic, Google Gemini, Hugging Face) and demonstrates how to work with both paid and free models. Additionally, it covers the use of embedding models for both single sentences and larger documents, along with examples of open-source embedding models.